

Projet - UE Outils pour la Statistique

Léo GAUTHIER

Partie 1 : Bootstrap

```
heart = read.csv("heart.csv")
```

On veut proposer un modèle de régression logistique pour expliquer la variable `target`. On commence par vérifier les NA :

```
colSums(is.na(heart))
```

age	sex	cp	trestbps	chol	fbs	restecg	thalach
0	0	0	0	0	0	0	0
exang	oldpeak	slope	ca	thal	target		
0	0	0	0	0	0		

On s'intéresse également aux statistiques descriptives :

```
describe(heart)
```

	vars	n	mean	sd	median	trimmed	mad	min	max	range	skew
age	1	303	54.37	9.08	55.0	54.54	10.38	29	77.0	48.0	-0.20
sex	2	303	0.68	0.47	1.0	0.73	0.00	0	1.0	1.0	-0.78
cp	3	303	0.97	1.03	1.0	0.86	1.48	0	3.0	3.0	0.48
trestbps	4	303	131.62	17.54	130.0	130.44	14.83	94	200.0	106.0	0.71
chol	5	303	246.26	51.83	240.0	243.49	47.44	126	564.0	438.0	1.13
fbs	6	303	0.15	0.36	0.0	0.06	0.00	0	1.0	1.0	1.97
restecg	7	303	0.53	0.53	1.0	0.52	0.00	0	2.0	2.0	0.16
thalach	8	303	149.65	22.91	153.0	150.98	22.24	71	202.0	131.0	-0.53
exang	9	303	0.33	0.47	0.0	0.28	0.00	0	1.0	1.0	0.74

oldpeak	10	303	1.04	1.16	0.8	0.86	1.19	0	6.2	6.2	1.26
slope	11	303	1.40	0.62	1.0	1.46	1.48	0	2.0	2.0	-0.50
ca	12	303	0.73	1.02	0.0	0.54	0.00	0	4.0	4.0	1.30
thal	13	303	2.31	0.61	2.0	2.36	0.00	0	3.0	3.0	-0.47
target	14	303	0.54	0.50	1.0	0.56	0.00	0	1.0	1.0	-0.18

	kurtosis	se
age	-0.57	0.52
sex	-1.39	0.03
cp	-1.21	0.06
trestbps	0.87	1.01
chol	4.36	2.98
fbs	1.88	0.02
restecg	-1.37	0.03
thalach	-0.10	1.32
exang	-1.46	0.03
oldpeak	1.50	0.07
slope	-0.65	0.04
ca	0.78	0.06
thal	0.25	0.04
target	-1.97	0.03

On dispose donc de 303 observations, et aucune observation n'est manquante.

1. Intervalles de confiance

La régression logistique est un modèle linéaire généralisé qui consiste à expliquer la probabilité $p_i = \mathbb{P}(Y_i = 1 | X_i^{(1)}, \dots, X_i^{(p)})$, avec dans notre cas $Y_i = \text{target}$ et $p = 13$. Le modèle s'écrit :

$$\text{logit}(p_i) = \beta_0 + \sum_{j=1}^{13} \beta_j X_i^{(j)}$$

On fait la régression logistique :

```
rlog = glm(target~., data=heart, family=binomial)
beta_hat = rlog$coefficients
summary(rlog)
```

Call:

```
glm(formula = target ~ ., family = binomial, data = heart)
```

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)	
(Intercept)	3.450472	2.571479	1.342	0.179653	
age	-0.004908	0.023175	-0.212	0.832266	
sex	-1.758181	0.468774	-3.751	0.000176	***
cp	0.859851	0.185397	4.638	3.52e-06	***
trestbps	-0.019477	0.010339	-1.884	0.059582	.
chol	-0.004630	0.003782	-1.224	0.220873	
fbs	0.034888	0.529465	0.066	0.947464	
restecg	0.466282	0.348269	1.339	0.180618	
thalach	0.023211	0.010460	2.219	0.026485	*
exang	-0.979981	0.409784	-2.391	0.016782	*
oldpeak	-0.540274	0.213849	-2.526	0.011523	*
slope	0.579288	0.349807	1.656	0.097717	.
ca	-0.773349	0.190885	-4.051	5.09e-05	***
thal	-0.900432	0.290098	-3.104	0.001910	**

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

Null deviance: 417.64 on 302 degrees of freedom
 Residual deviance: 211.44 on 289 degrees of freedom
 AIC: 239.44

Number of Fisher Scoring iterations: 6

On estime ensuite des coefficients par bootstrap pour β_{age} :

```
set.seed(123)
n = dim(heart)[1]
B=1000
coeff_boot = c()
for(i in 1:B){
  index = sample(1:n, n, replace=TRUE)
  boot_data = slice(heart, index)
  coeff_boot[i] = glm(target~., data=boot_data,
                      family=binomial)$coefficients["age"]
}
```

Pour fournir un intervalle de confiance pour β_{age} , on implémente 2 manières différentes. On travaille au niveau de confiance $1 - \alpha = 95\%$.

Intervalle de confiance des percentiles

Cet intervalle de confiance est donné par la formule suivante :

$$IC_{\text{perc}}(1 - \alpha) = [\hat{\beta}_{age,(\lceil B\alpha/2 \rceil)}^*; \hat{\beta}_{age,(\lceil B(1-\alpha/2) \rceil)}^*]$$

où les $\hat{\beta}_{age,(1)}^* \leq \dots \leq \hat{\beta}_{age,(B)}^*$ sont les B estimations bootstrap obtenues du coefficient β_{age} .

```
perCI = function(x, alpha){
  x=sort(x)
  return(list(inf=x[ceiling(length(x)*alpha/2)],
              sup=x[ceiling(length(x)*(1-alpha/2))]))
}

(CI_1 = perCI(coeff_boot, 0.05))
```

```
$inf
[1] -0.04877269
```

```
$sup
[1] 0.03725405
```

```
# Longueur de l'intervalle
CI_1$sup - CI_1$inf
```

```
[1] 0.08602674
```

Le problème vu en cours de cette méthode est qu'elle considère la loi empirique des estimations bootstrap $\hat{\beta}_{age}^*$ comme étant la loi de l'estimateur $\hat{\beta}_{age}$.

Intervalle de confiance basé sur les erreurs

La deuxième méthode vue en cours est de d'abord proposer un intervalle de confiance bootstrap de l'erreur $\hat{\beta}_{age} - \beta_{age}$, dont les réalisations bootstrap sont $\hat{\beta}_{age,b} - \hat{\beta}_{age}$, donné par:

$$[\hat{\beta}_{age,(\lceil B\alpha/2 \rceil)}^*; \hat{\beta}_{age,(\lceil B(1-\alpha/2) \rceil)}^*]$$

L'intervalle de confiance pour β_{age} est alors :

$$IC(1 - \alpha) = [2\hat{\beta}_{age} - \hat{\beta}_{age,(\lceil B(1-\alpha/2) \rceil)}^*; 2\hat{\beta}_{age} - \hat{\beta}_{age,(\lceil B\alpha/2 \rceil)}^*]$$

```
npCI = function(x, alpha){  
  CI.sup = 2*mean(x) - sort(x)[ceiling(B*alpha/2)]  
  CI.inf = 2*mean(x) - sort(x)[ceiling(B*(1-alpha/2))]  
  return(list(inf=CI.inf, sup=CI.sup))  
}  
  
(CI_2 = npCI(coeff_boot, 0.05))
```

```
$inf  
[1] -0.04785805
```

```
$sup  
[1] 0.03816869
```

```
# Longueur de l'intervalle  
CI_2$sup - CI_2$inf
```

```
[1] 0.08602674
```

Le fait que la longueur soit identique est parfaitement logique, on utilise les mêmes quantiles mais on s'est juste déplacé pour être centré autour de $\hat{\beta}_{age}$.

On a vu une autre méthode en cours, l'intervalle de confiance du bootstrap standardisé où l'on considère la loi de la statistique de Student plutôt que les quantiles empiriques de $\hat{\beta}^*$. On se contentera ici des 2 méthodes précédentes.

On préférera l'intervalle de confiance obtenu par la deuxième méthode. En effet, au lieu d'approximer la loi de $\hat{\beta}_{age}$ par la loi empirique obtenue par bootstrap, on approxime la loi de son erreur $\hat{\beta}_{age} - \beta_{age}$, bien que l'on sait que la loi empirique de $\hat{\beta}_{age}^*$ est proche de la loi de $\hat{\beta}_{age}$ lorsque n est grand.

2. Test d'hypothèse

Puisque $0 \in IC_{\text{perc}}(1 - \alpha)$, il n'est pas nécessaire de faire un test pour tester la significativité de $\hat{\beta}_{age}$, on peut d'ores et déjà conclure à sa non-significativité.

Par acquis de conscience, on propose tout de même un test bootstrap.

On veut donc tester : $\mathcal{H}_0 : \hat{\beta}_{age} = 0$ contre $\mathcal{H}_1 : \hat{\beta}_{age} \neq 0$.

```
set.seed(123)
(p_val_boot2= mean(abs(coeff_boot) >= abs(beta_hat["age"])))
```

[1] 0.824

Dans le cours de GLM (1), on a vu que le meilleur test pour la sélection de variables est le LRT car c'est le plus puissant.

Sous $\mathcal{H}_0$, on sait que :

$$LR \xrightarrow[n \rightarrow +\infty]{\mathcal{L}} \chi^2(1)$$

où LR est la statistique de test, donnée par :

$$LR = 2(\ell(\hat{\beta}) - \ell(\hat{\beta}_0))$$

où $\ell(\cdot)$ est la log-vraisemblance du modèle, $\hat{\beta}$ le vecteur des coefficients estimés dans le modèle complet (donc sous $\mathcal{H}_1$) et $\hat{\beta}_0$ le vecteur des coefficients estimés dans le modèle restreint (donc sous $\mathcal{H}_0$). Dans la suite, on travaillera au seuil de significativité $\alpha = 0.05$.

On sait que dans le cadre d'une procédure de test basée sur le bootstrap, il est nécessaire de pouvoir simuler sous $\mathcal{H}_0$. Cela signifie donc de se placer dans un cadre où l'effet de la variable **age** n'existe pas. Il faut donc non seulement supprimer **age** du jeu de données, mais également re-simuler la variable réponse **target**. En effet, l'effet de l'âge sur la réponse reste présent, même s'il n'est pas significatif il sera "résiduel" (dans les corrélations et dépendances entre **age** et les autres variables). Pour avoir une réponse **target** où l'âge n'a aucun effet (et n'est donc pas significatif), il faut re-simuler la réponse avec le jeu de données restreint.

La procédure consiste à :

1. Estimer le modèle restreint sans **age** sur l'échantillon original
2. Simuler les nouvelles réponses selon les probabilités prédites par ce modèle
3. Appliquer le bootstrap sur le jeu de données restreint avec les nouvelles réponses

4. Calculer la statistique de test à chaque itération bootstrap

```
heart_noage = heart %>%
  select(-age)

set.seed(123)

# Partie observée :
rlog_0 = glm(target~., data=heart_noage, family=binomial)
rlog = glm(target~., data=heart, family=binomial)
logv0 = as.numeric(logLik(rlog_0))
logv = as.numeric(logLik(rlog))
LR_obs = 2*(logv - logv0)

# Partie bootstrap
logv0_boot = c()
logv_boot = c()
LR_boot = c()

for(b in 1:B){
  index = sample(1:n, n, replace=TRUE)

  # On re-simule target sur le jeu de données restreint
  y_noage = rbinom(n = n, size = 1,
    prob = predict(rlog_0, type = "response"))

  heart_b = heart
  heart_b$target = y_noage

  boot_heart <- slice(heart_b, index)

  rlog_0 = glm(target ~.-age, data = boot_heart, family = binomial)
  rlog_1 = glm(target ~ ., data = boot_heart, family = binomial)
  logv0_boot[b] = as.numeric(logLik(rlog_0))
  logv_boot[b] = as.numeric(logLik(rlog_1))
  LR_boot[b] = 2*(logv_boot[b]-logv0_boot[b])
}
cat("La p-valeur obtenue est :", p_val_boot = mean(LR_boot >= LR_obs), "\n")
```

La p-valeur obtenue est : 0.202

On obtient une $p\text{-val} > \alpha$, donc on ne rejette pas $\mathcal{H}_0$, on considère que l'âge n'a pas un effet significatif sur le risque d'arrêt cardiaque.

Conclusion

On a mis en place deux méthodes similaires d'intervalle de confiance par bootstrap pour le coefficient β_{age} , qui contienne chacun 0.

On a ensuite testé la significativité de l'estimation de ce coefficient par un test du rapport de vraisemblance. Ce test n'était cependant pas indispensable étant donné que 0 est contenu dans les intervalles. Ce test ne permet pas de mettre en lumière d'effet significatif de la variable age , ce qui est cohérent avec les intervalles de confiance.

Partie 2 : Algorithmes EM et Stochastic EM

1. Contexte du travail

Le travail exposé par M. BORDES et M. CHAUVEAU est une application de l'algorithme EM et de ses variantes (EM stochastique, EM semi-paramétrique) dans le cadre de données de survie. Il propose alors un algorithme EM dans le cadre d'un modèle de mélange en présence de données censurées.

2. Notations

Dans le cadre de l'EM paramétrique pour données complètes, on dispose d'un modèle de mélange à m composantes et n observations. On note :

- $X = (X_1, \dots, X_n) \in \mathbb{R}^n$ les variables aléatoires représentant les durées de vie observées, qu'on suppose *iid* et de densité $X \sim g(x|\theta) = \sum_{j=1}^m \lambda_j f_j(x)$, où $f_z(\cdot) \in \mathcal{F} = \{f(\cdot|\xi), \xi \in \Xi\}$, où $\lambda_j > 0$, $\sum_{j=1}^m \lambda_j = 1$.
- $C = (C_1, \dots, C_n) \in \mathbb{R}^n$ les n temps de censure, qu'on suppose *iid*, et de densité q .

De manière habituelle en analyse de survie, certaines observations sont manquantes. Dans cet exposé, on est dans le cadre d'une censure aléatoire à droite non-informative : on dispose des variables aléatoires $T = \min(X, C)$ et non des variables X , telles que $T \perp C$. On note également $D = \mathbb{1}_{\{X \leq C\}}$ l'indicateur de censure.

On dispose donc des observations $(t, d) = ((t_1, \dots, t_n), (d_1, \dots, d_n))$.

On note également

- $Z = (Z_1, \dots, Z_n)$ la variable aléatoire qualitative latente non-observée, marquant ici l'appartenance d'une observation à une des m classes du mélange (à valeurs dans $\{1, \dots, m\}$). Il est supposé que $Z \sim \text{Mult}(1, \lambda)$, où $\lambda = (\lambda_1, \dots, \lambda_m)$ (c'est-à-dire $\mathbb{P}(Z_i = j) = \lambda_j$).

M. BORDES et M. CHAUVEAU introduisent deux niveaux de données complètes : (X, Z) et (T, D, Z) . La différence entre les deux réside dans l'expression de leur densité jointe.

On se place dans le cadre paramétrique, donc les paramètres régissant le modèle θ est l'ensemble des paramètres de la loi de X (ici, $\xi = (\xi_1, \dots, \xi_m)$) et des paramètres gouvernant Z (les probabilités $\mathbb{P}(Z = j)$, où $j = 1, \dots, m$, ici λ). Donc $\theta = (\lambda_1, \dots, \lambda_m, \xi_1, \dots, \xi_m)$.

3. La slide 21

On se place dans le cadre d'un algorithme EM pour un modèle de mélange à deux composantes exponentielles.

L'algorithme EM consiste, à chaque itération k , à déterminer une valeur $\theta^{(k+1)}$ telle que $\Delta(\theta^{(k+1)}, \theta^{(k)}) := \ell(\theta^{(k+1)}) - \ell(\theta^{(k)}) \geq 0$, et maximisant cette quantité. Cependant, on ne sait pas maximiser $\ell(\theta^{(k)})$, donc on cherche à déterminer une fonction $\delta(\theta^{(k+1)}|\theta^{(k)})$ vérifiant $\Delta(\theta^{(k+1)}, \theta^{(k)}) \geq \delta(\theta^{(k+1)}|\theta^{(k)})$ et $\delta(\theta^{(k)}|\theta^{(k)}) = 0$.

D'après la slide 20, on dispose pour la fonction $\delta(\theta|\theta^{(k)})$, si $\theta = (\lambda, \xi)$:

$$\delta(\theta|\theta^{(k)}) = Q(\theta|\theta^{(k)}) = \mathbb{E}[\ell^c(t, d, Z|\theta)|t, d, \theta^{(k)}]$$

où $\ell^c(t, d, Z|\theta) = \log f^c(t, d, Z|\theta)$.

Or, par indépendance des composantes (X, Z) et linéarité de l'espérance, on peut décomposer cette espérance en :

$$\delta(\theta|\theta^{(k)}) = \sum_{i=1}^n \mathbb{E}[\ell^c(t_i, d_i, Z_i|\theta)|t_i, d_i, \theta^{(k)}]$$

En introduisant les $p_{ij}^k = \mathbb{P}(Z_i = j|t_i, d_i, \theta^{(k)})$, on obtient (à une constante indépendante de θ près, qui n'interviendra donc pas dans le problème de maximisation) :

$$\delta(\theta|\theta^{(k)}) = \sum_{i=1}^n \sum_{j=1}^m \mathbb{P}(Z_i = j|t_i, d_i, \theta^{(k)}) \ell^c(t_i, d_i, Z_i = j|\theta) = \sum_{i=1}^n \sum_{j=1}^m p_{ij}^k \ell^c(t_i, d_i, Z_i = j|\theta)$$

Il suffit désormais d'injecter la vraisemblance, et on obtient alors :

$$\delta(\theta|\theta^{(k)}) = \sum_{i=1}^n \sum_{j=1}^m p_{ij}^k (\log [(\lambda_j f(t_i|\xi_j))^{d_i} (\lambda_j \bar{F}(t_i|\xi_j))^{1-d_i}] + \log [\bar{Q}(t_i)^{d_i} q(t_i)^{1-d_i}])$$

La dernière partie étant indépendante de θ , elle n'interviendra pas dans le problème de maximisation de la fonction $\delta(\theta|\theta^{(k)})$.

L'algorithme EM comme détaillé par M. BORDES consiste donc à choisir $\theta^{(k+1)}$ comme étant :

$$\theta^{(k+1)} = \arg \max_{\theta \in \Theta} \sum_{i=1}^n \sum_{j=1}^m p_{ij}^k \log [(\lambda_j f(t_i|\xi_j))^{d_i} (\lambda_j \bar{F}(t_i|\xi_j))^{1-d_i}]$$

et après simplification :

$$\theta^{(k+1)} = \arg \max_{\theta \in \Theta} \sum_{i=1}^n \sum_{j=1}^m p_{ij}^k (\log \lambda_j + d_i \log f(t_i|\xi_j) + (1-d_i) \log \bar{F}(t_i|\xi_j))$$

On a supposé que les durées de vie sont exponentielles, i.e. $X_i|Z_i = j \sim \mathcal{E}(\xi_j)$, $i \in \{1, \dots, n\}$, on a donc $f(t_i|\xi_j) = \xi_j \exp(-\xi_j t_i)$ et $\bar{F}(t_i|\xi_j) = \exp(-\xi_j t_i)$. Donc on obtient

$$\theta^{(k+1)} = \arg \max_{\theta \in \Theta} \sum_{i=1}^n \sum_{j=1}^m p_{ij}^k (\log \lambda_j + d_i \log \xi_j - \xi_j t_i)$$

Il suffit maintenant de dériver cette formule pour obtenir les formules de mise à jour pour obtenir $\lambda_j^{(k+1)}$ et $\xi_j^{(k+1)}$. On optimise sous la contrainte $\sum_{j=1}^m \lambda_j = 1$ par définition de la loi multinomiale. On pose le lagrangien

$$\mathcal{L}(\lambda, \xi|\alpha) = \sum_{i=1}^n \sum_{j=1}^m p_{ij}^k (\log \lambda_j + d_i \log \xi_j - \xi_j t_i) - \alpha (\sum_{j=1}^m \lambda_j - 1)$$

On dérive, pour j fixé dans $\{1, \dots, m\}$, par rapport à α , λ_j et ξ_j . On obtient alors les formules de mise à jour suivantes :

$$\lambda_j^{(k+1)} = \frac{1}{n} \sum_{i=1}^n p_{ij}^k$$

car $\sum_{j=1}^m p_{ij}^k = 1$ et

$$\xi_j^{(k+1)} = \frac{\sum_{i=1}^n p_{ij}^k d_i}{\sum_{i=1}^n p_{ij}^k t_i}$$

qui correspondent donc bien aux formules de mise à jour proposées dans la slide 21.

4. Implémentation de l'EM paramétrique

On simule selon un modèle de mélange de deux lois exponentielles (donc $j \in \{1, 2\}$) de paramètres respectifs $\xi_1 = 1$, $\xi_2 = 0.2$, et avec $\lambda_1 = \frac{1}{3}$. Il faut également prendre en compte la censure dans la simulation. Pour pouvoir respecter la reproductibilité des résultats, on aimerait pouvoir censurer un certain pourcentage des données comme le fait M. BORDES.

```
# On crée la fonction qui simule des données censurées

mélange_expo_cens = function(lambda, rate, taille.ech, rate.cens){
  K=length(lambda)
  z=sample(c(1:K), taille.ech, prob=lambda, replace=TRUE)
  x = rexp(taille.ech, rate = rate[z])

  C = rexp(taille.ech, rate=rate.cens)

  x.cens = pmin(x, C) #compare chaque x[i] et C entre eux
  d = as.numeric(x<=C) #sinon renvoie booléen

  return(data.frame(x.cens = x.cens, d=d))
}
```

L'argument `rate_cens` fait référence au paramètre de la variable aléatoire C_i de censure, qui est supposée non informative.

Dans l'exemple de M. BORDES, il suppose $C_i \sim \mathcal{E}(\mu)$, et il utilise une censure de 30%. Déterminons μ :

On connaît la loi de $X_i|Z_i = j$, mais pas la loi de X_i , on ne peut donc pas résoudre immédiatement $\mathbb{P}(X_i > C_i) = 0.3$. On utilise les probabilités totales :

$$\mathbb{P}(X_i > C_i) = \mathbb{P}(X_i > C_i|Z_i = 1)\mathbb{P}(Z_i = 1) + \mathbb{P}(X_i > C_i|Z_i = 2)\mathbb{P}(Z_i = 2)$$

Donc :

$$\mathbb{P}(X_i > C_i) = \lambda_1 \mathbb{P}(X_i > C_i|Z_i = 1) + \lambda_2 \mathbb{P}(X_i > C_i|Z_i = 2)$$

Et donc, pour $j \in \{1, 2\}$,

$$\mathbb{P}(X_i > C_i|Z_i = j) = \int_0^{+\infty} \mathbb{P}(X_i > C_i|Z_i = j, X_i = x) f(x|\xi_j) dx$$

C_i étant non-informative, on obtient $\mathbb{P}(X_i > C_i | Z_i = j) = \int_0^{+\infty} \mathbb{P}(C_i < x) f(x | \xi_j) dx$. Donc

$$\mathbb{P}(X_i > C_i | Z_i = j) = \int_0^{+\infty} [1 - e^{-\mu x}] \xi_j e^{-x \xi_j} dx = \frac{\mu}{\xi_j + \mu}.$$

Si on se place dans le même cadre que l'exemple de la slide 21, on obtient :

$$\mathbb{P}(X_i > C_i) = \frac{\mu}{3(\mu + 1)} + \frac{2\mu}{3(\mu + 0.2)} = 0.3$$

Ce qui donne $\mu \approx 0.1294$. Pour 34.7% de censure, on trouve $\mu \approx 0.1637$. Travaillons avec $\mu = 15\%$ pour pouvoir comparer les résultats obtenus.

```
set.seed(98)
sim1 = melange_expo_cens(lambda=c(1/3, 2/3), rate=c(1,0.2), rate.cens=0.15,
                        taille.ech=200)

sim2 = melange_expo_cens(lambda=c(1/3, 2/3), rate=c(1,0.2), rate.cens=0.15,
                        taille.ech=1000)
```

Implémentons maintenant l'algorithme EM.

On commence par préparer la fonction de vraisemblance :

```
# On commence par implémenter la fonction de vraisemblance censurée

vrais.comp=function(x, d, lambda, rate){
  n=length(x) ; K=length(lambda)
  v=matrix(data=NA,ncol=K, nrow=n)
  for(k in 1:K){
    v[,k] = lambda[k] * ifelse(d==1, dexp(x, rate=rate[k]),
                               pexp(x, rate = rate[k], lower.tail = FALSE))
  }
  l=c()
  for(i in 1:n){
    l[i] = sum(v[i,])
  }
  return(sum(log(l)))
}
```

```

# Algorithme EM pour données censurées :

my_em_exp_censored = function(x, d, niter, start, trace=TRUE, verbose){
  n = length(x)
  lambda = start$lambda
  rate = start$rate
  K = length(lambda)
  eta = matrix(NA, nrow=n, ncol=K)
  theta = NULL
  log.vrais=c()
  iterations = 1:niter

  if(trace){
    theta=matrix(NA,niter,2*K)
  }

  for(iter in 1:niter){

    if(trace){
      theta[iter,] = c(lambda,rate)
    }

    # Mise à jour des  $p_{ij}^k$ 
    for(k in 1:K){
      eta[,k] = lambda[k] * (d*dexp(x, rate=rate[k]) +
        (1-d)*pexp(x, rate = rate[k], lower.tail = FALSE))
    }

    eta=eta/rowSums(eta)

    # Mise à jour des paramètres

    lambda = colMeans(eta)

    for(k in 1:K){
      rate[k] = (sum(eta[,k]*d) / sum(eta[,k]*x))
    }

    log.vrais[iter]=vrais.comp(x,d,lambda,rate)

    if(verbose) print(c(iter,lambda,rate))
  }
}

```

```

}
return(list(lambda=lambda, rate=rate, theta=theta,
            log.vrais=log.vrais, iteration = iterations))
}

```

```

set.seed(98)
# Point de départ de l'algorithme, à faire varier
start = list(lambda=c(1/3,2/3), rate=c(1,0.2))

# Application de l'algorithme EM
res1 = my_em_exp_censored(x=sim1[,1], d=sim1[,2], niter=1000,
                          start=start, verbose=FALSE)
res2 = my_em_exp_censored(x=sim2[,1], d=sim2[,2], niter=1000,
                          start=start, verbose=FALSE)

```

On a $\theta_{[1]} = \lambda_1$, $\theta_{[3]} = \xi_1$, $\theta_{[4]} = \xi_2$.

Les résultats sont :

```
res1$lambda ; res1$rate
```

```
[1] 0.1521803 0.8478197
```

```
[1] 2.8096327 0.2476497
```

```
res2$lambda ; res2$rate
```

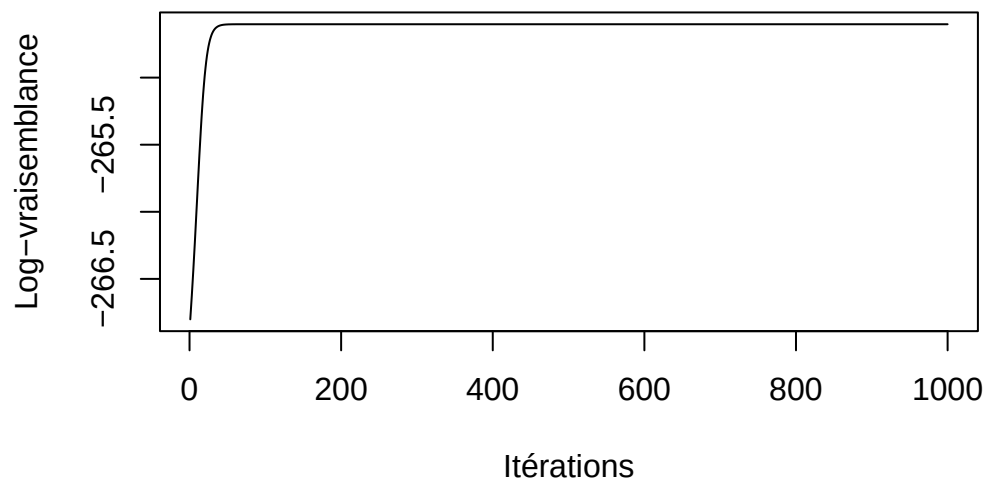
```
[1] 0.3048456 0.6951544
```

```
[1] 1.0478881 0.2241972
```

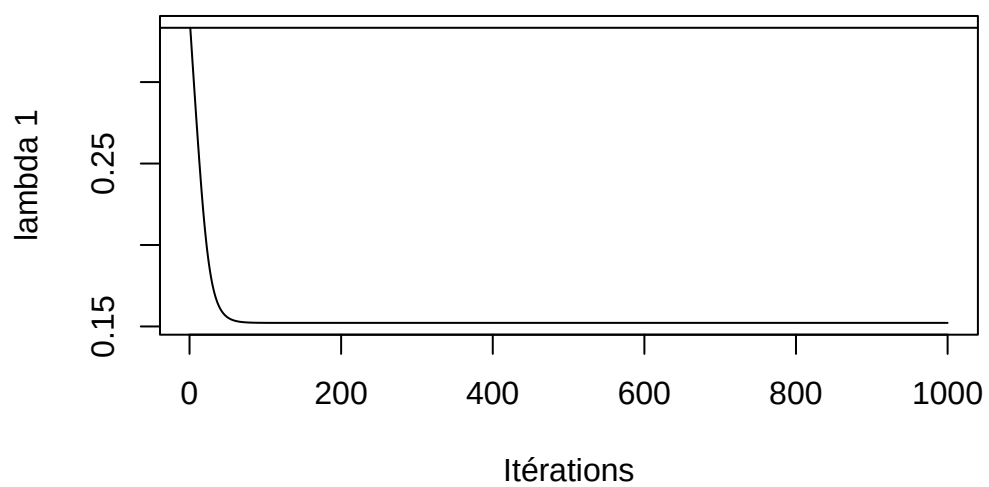
La simulation avec un échantillon plus grand permet naturellement d'améliorer les estimations.

Voici quelques représentations graphiques des résultats et des trajectoires des estimations de cet algorithme :

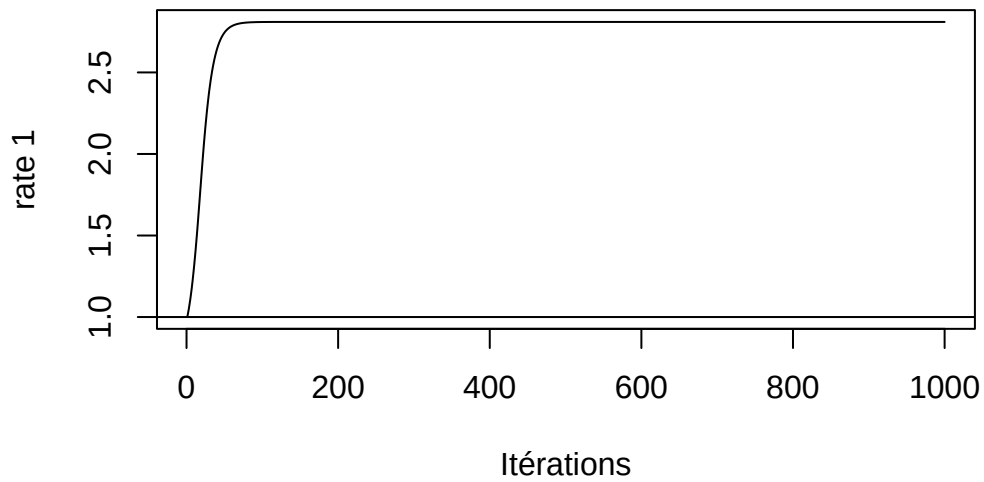
```
plot(res1$log.vrais, type="l", xlab="Itérations", ylab="Log-vraisemblance")
```



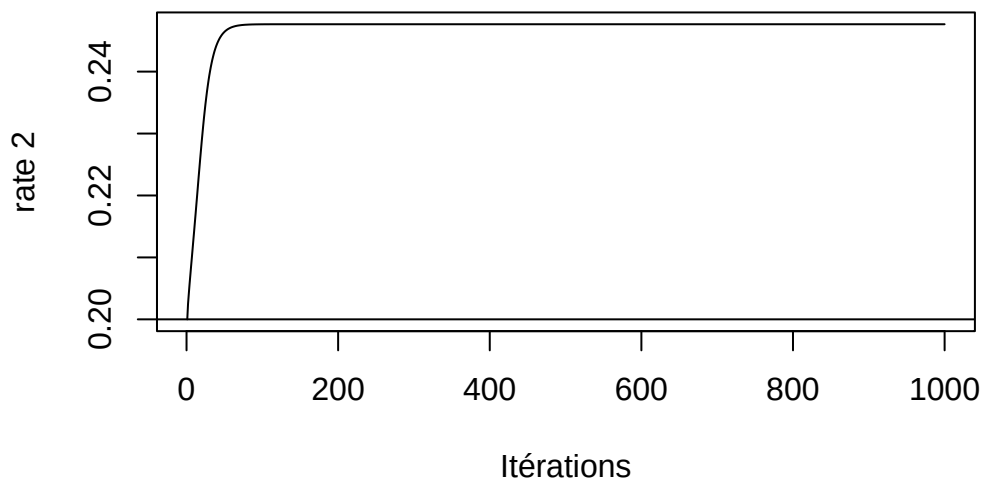
```
plot(res1$iteration, res1$theta[,1], xlab="Itérations",  
      ylab="lambda 1", type="l")  
abline(h=1/3)
```



```
plot(res1$iteration, res1$theta[,3], xlab="Itérations",  
     ylab="rate 1", type="l")  
abline(h=1)
```



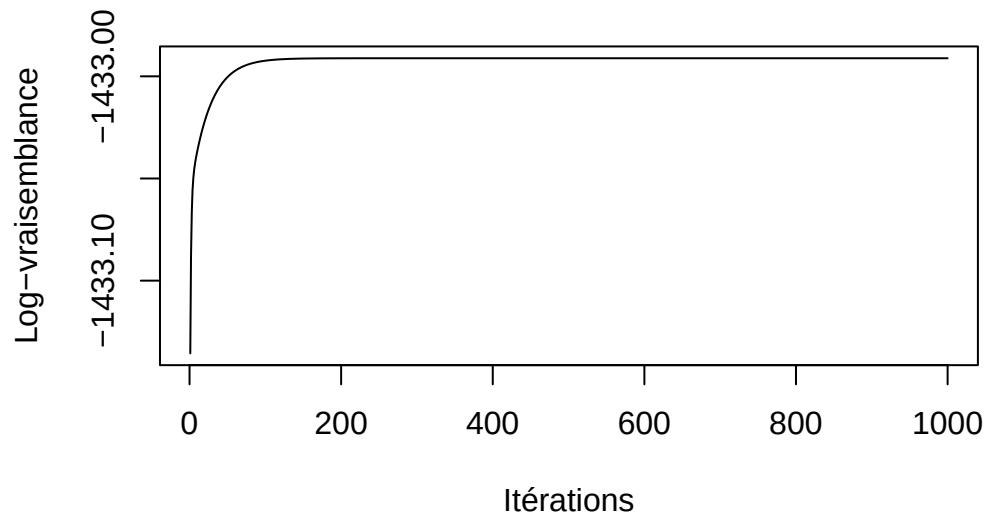
```
plot(res1$iteration, res1$theta[,4], xlab="Itérations",  
     ylab="rate 2", type="l")  
abline(h=0.2)
```

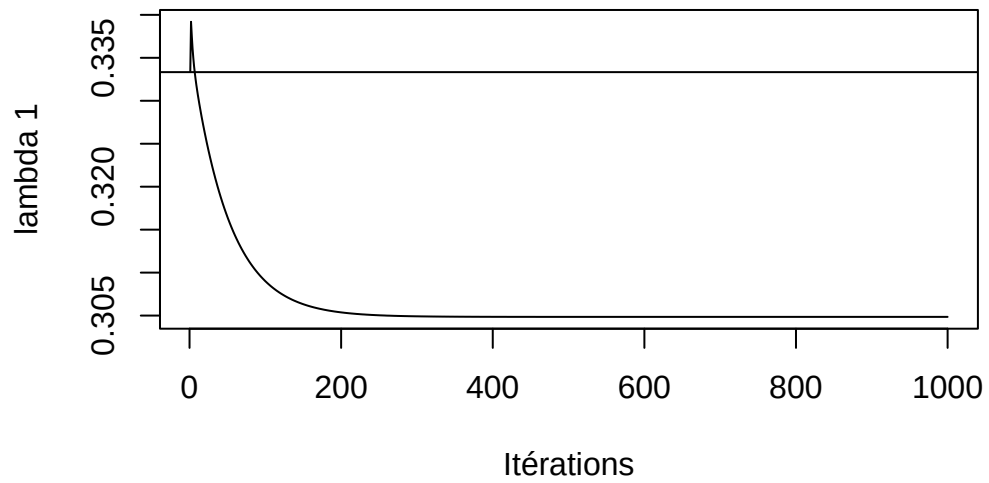
On voit que la log-vraisemblance se stabilise après très peu d'itérations.

Avec le deuxième jeu simulé, on a :

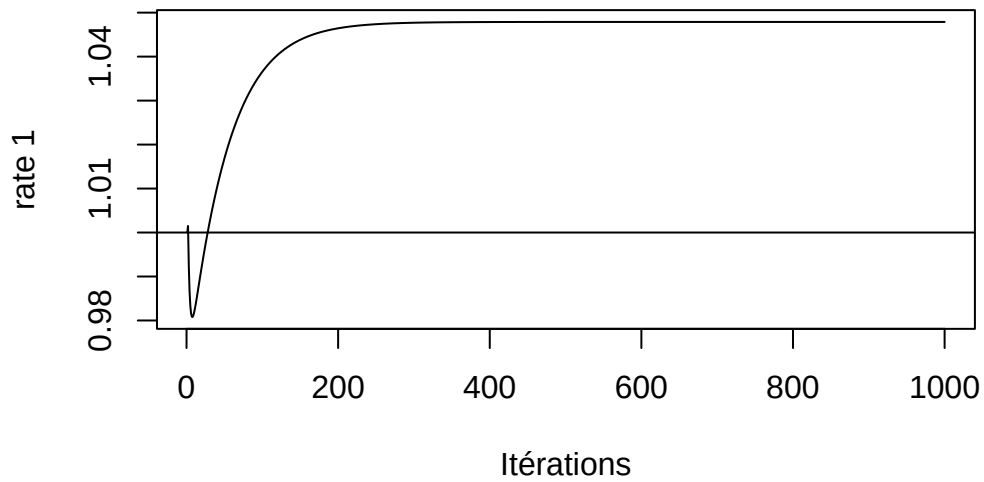
```
plot(res2$log.vrais, type="l", xlab="Itérations", ylab="Log-vraisemblance")
```



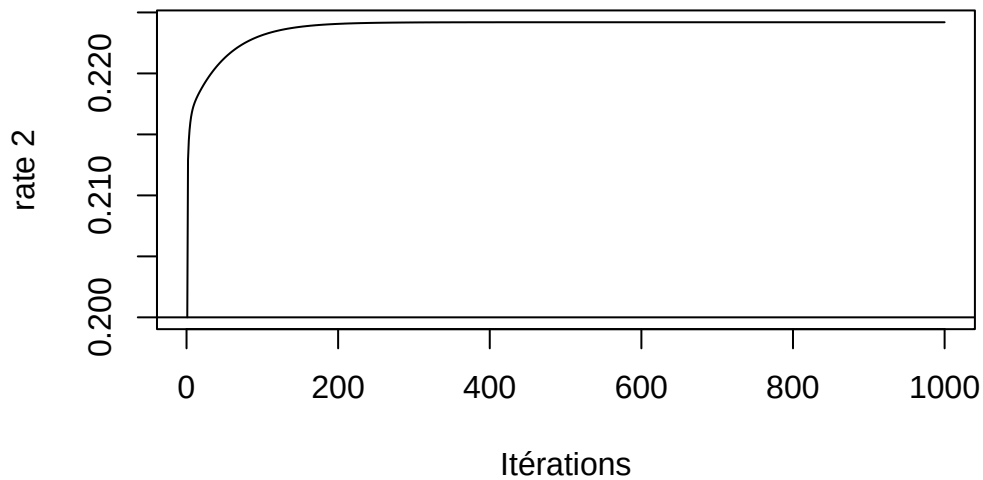
```
plot(res2$iteration, res2$theta[,1], xlab="Itérations",  
      ylab="lambda 1", type="l")  
abline(h=1/3)
```



```
plot(res2$iteration, res2$theta[,3], xlab="Itérations",
     ylab="rate 1", type="l")
abline(h=1)
```



```
plot(res2$iteration, res2$theta[,4], xlab="Itérations",
     ylab="rate 2", type="l")
abline(h=0.2)
```



On voit aussi bien que la log-vraisemblance s'est très vite stabilisée au fur et à mesure des itérations.

On crée également le même graphique que M. BORDES dans sa présentation. Pour le premier jeu simulé :

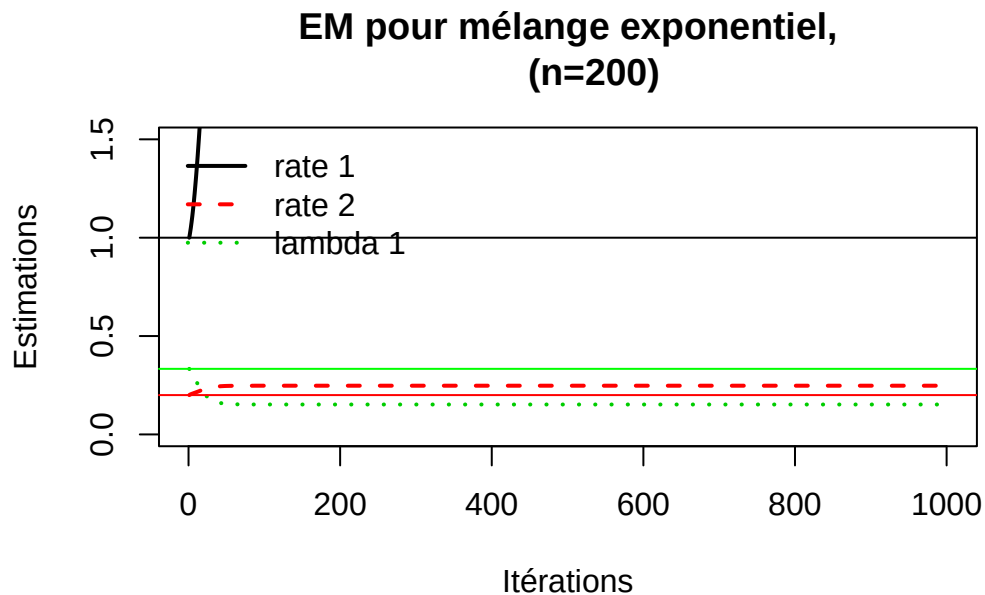
```
plot(1:nrow(res1$theta), res1$theta[, 3], type = "l", lwd = 2,
     ylim = c(0, 1.5),
     xlab = "Itérations",
     ylab = "Estimations",
     main = "EM pour mélange exponentiel,
            (n=200)")

lines(1:nrow(res1$theta), res1$theta[, 4], col = "red", lty = 2, lwd = 2)
lines(1:nrow(res1$theta), res1$theta[, 1], col = "green3", lty = 3, lwd = 2)

abline(h = 1)
abline(h=.2, col = "red")
abline(h=1/3, col="green")

legend("topleft",
      legend = c("rate 1", "rate 2", "lambda 1"),
      col = c("black", "red", "green3"),
      lty = c(1, 2, 3),
```

```
lwd = 2,
bty = "n")
```



Et pour le deuxième jeu simulé :

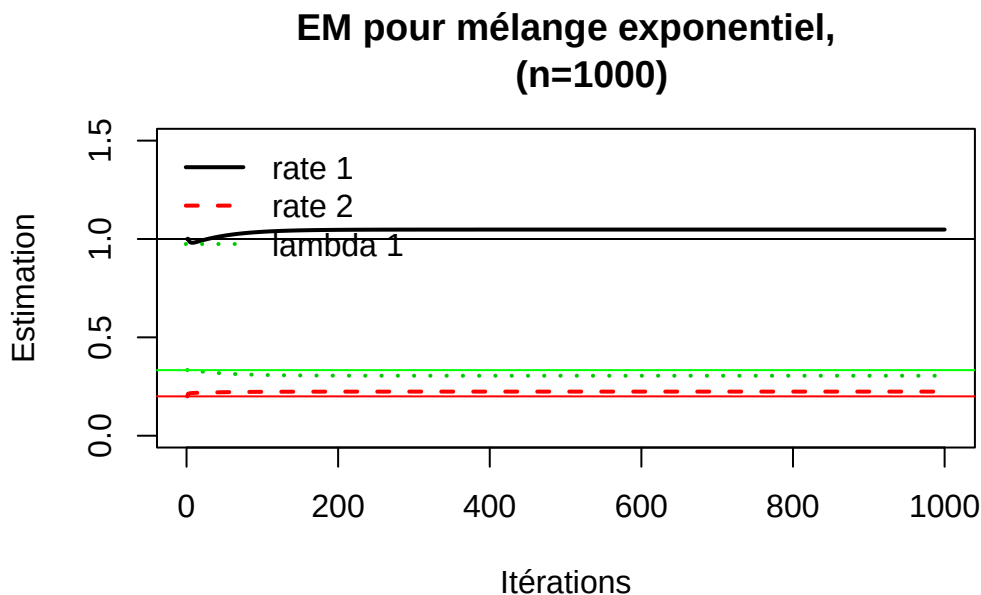
```
plot(1:nrow(res2$theta), res2$theta[, 3], type = "l", lwd = 2,
     ylim = c(0, 1.5),
     xlab = "Itérations",
     ylab = "Estimation",
     main = "EM pour mélange exponentiel,
            (n=1000)")

lines(1:nrow(res2$theta), res2$theta[, 4], col = "red", lty = 2, lwd = 2)
lines(1:nrow(res2$theta), res2$theta[, 1], col = "green3", lty = 3, lwd = 2)

abline(h = 1)
abline(h=.2, col = "red")
abline(h=1/3, col="green")

legend("topleft",
      legend = c("rate 1", "rate 2", "lambda 1"),
      col = c("black", "red", "green3"),
      lty = c(1, 2, 3),
```

```
lwd = 2,
bty = "n")
```



Pour illustrer l'impact du point de départ, on peut reprendre le deuxième jeu de données simulé avec différents points de départ :

```
set.seed(98)
# On définit 3 points de départ différents

start2 = list(lambda=c(2/3, 1/3), rate=c(1,0.2))
start3 = list(lambda=c(2/3, 1/3), rate=c(1.5, 0.5))
# Intéressant de générer un point de départ aléatoire
start4 = list(lambda=c(runif(1, 0.1, 0.9), runif(1, 0.1, 0.9)),
              rate = c(runif(1,0.1,1.5),runif(1,0.1,1.5)))

# On applique l'algorithme à chaque point de départ

res212 = my_em_exp_censored(x=sim2[,1], d=sim2[,2], niter=1000,
                           start=start2, verbose=FALSE)
res213 = my_em_exp_censored(x=sim2[,1], d=sim2[,2], niter=1000,
                           start=start3, verbose=FALSE)
res214 = my_em_exp_censored(x=sim2[,1], d=sim2[,2], niter=1000,
                           start=start4, verbose=FALSE)
```

```
# On les stocke dans une liste, ça sera utile après

res_sim2 = list(res2, res212, res213, res214)
```

Présentons les résultats sous forme de graphique. Je commence par créer une fonction `evol_logvr`, qui va permettre de plot immédiatement la log-vraisemblance des 4 starts différents à partir de la liste `res_sim2`.

```
colors = c("black","red","blue","green") # une couleur par start

evol_logvr = function(res, titre){

  logv_all = list(res[[1]]$log.vrais, res[[2]]$log.vrais,
                  res[[3]]$log.vrais, res[[4]]$log.vrais)

  ylim_all = range(unlist(logv_all))

  ltys = c(1, 2, 3, 4)

  plot(logv_all[[1]], type="l", col=colors[1], ylim = ylim_all,
        xlab="Itération",ylab="Log-vraisemblance",
        main = titre, lwd=2, las=1)

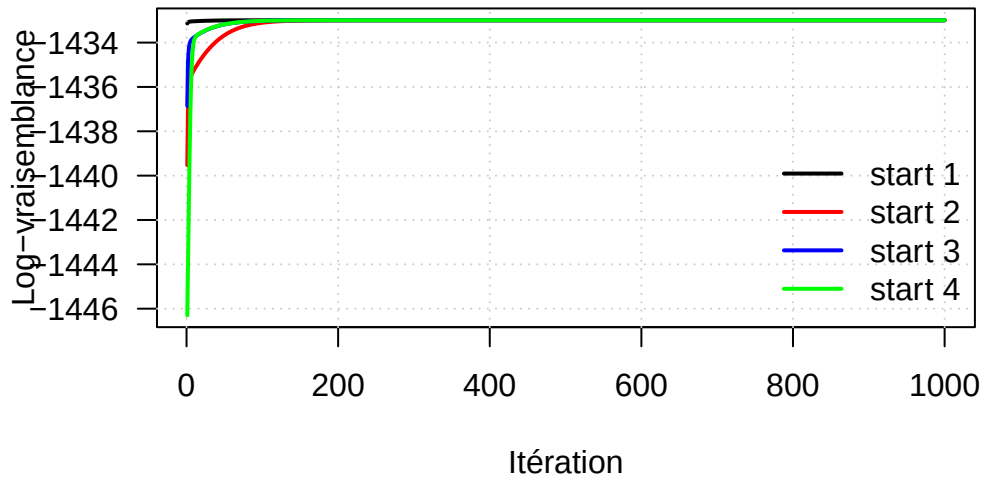
  grid(col="grey80", lty=3)

  for(i in 2:length(logv_all)){
    lines(logv_all[[i]], lwd=2, col=colors[i])
  }

  legend("bottomright",
        legend = c("start 1", "start 2", "start 3", "start 4"),
        col = colors, lwd=2, bty="n")
}

evol_logvr(res=res_sim2, titre="2er jeu de données (n=1000)")
```

2er jeu de données (n=1000)



On voit que la log-vraisemblance est croissante et converge très vite, en moins de 200 itérations dans la plupart des cas.

Illustrons également l'impact du point de départ en fonction des paramètres estimés.

J'ai tout d'abord régulièrement rencontré un problème dans mes tests : le ξ_1 était échangé avec le ξ_2 , et pareil pour λ_1 et λ_2 . C'est le label switching, qui est un problème récurrent dans les modèles de mélange ((2), section 1.14). Ce problème n'impacte pas la vraisemblance mais rend compliqué l'interprétation des paramètres finaux.

Une solution très simple et adaptée dans notre cas est l'utilisation d'une contrainte d'identifiabilité. Elle est adaptée car on connaît l'ordre à priori dans nos simulations : $\lambda_1 < \lambda_2$ et $\xi_1 > \xi_2$. On ré-organise juste les paramètres si ils sont inversés :

```
res_sim2 = lapply(res_sim2, function(r){
  o = order(r$lambda)
  r$lambda = r$lambda[o]
  r$rate = r$rate[o]

  if(!is.null(r$theta)){
    r$theta = t(apply(r$theta, 1, function(v){
      lambda = v[1:2]
      rate = v[3:4]
      o2 = order(lambda)
      c(lambda[o2], rate[o2])
    })
  )
})
```



```

    )))
  }

  return(r)
})

```

Maintenant, on peut proposer des graphiques illustrant les trajectoires des estimations. Commençons par les estimations des λ_j :

```

# Un type de ligne pour chaque paramètre :
ltys = c(1,2)

# Délimitation de la fenêtre graphique :
ylim_lam = range(unlist(lapply(res_sim2, function(r) r$theta[,1:2])))

# Le graphique :
plot(res_sim2[[1]]$theta[,1], type="l", col=colors[1], lty=ltys[1],
      ylim=ylim_lam, xlab="Itération", ylab="Rate",
      main="Trajectoire des lambdas")

grid(col="grey80", lty=3)

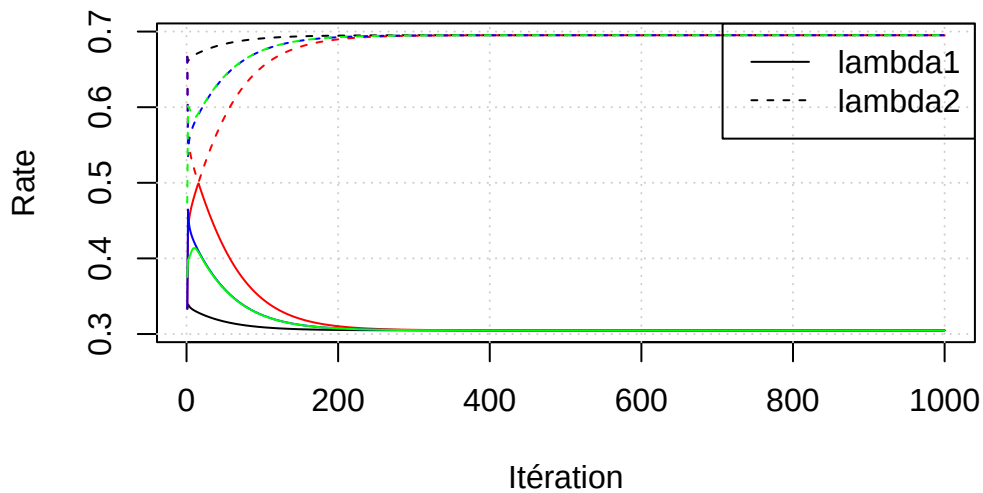
lines(res_sim2[[1]]$theta[,2], col=colors[1], lty=ltys[2])

for(i in 2:length(res_sim2)){
  lines(res_sim2[[i]]$theta[,1], col=colors[i], lty=ltys[1])
  lines(res_sim2[[i]]$theta[,2], col=colors[i], lty=ltys[2])
}

legend("topright", legend=c("lambda1","lambda2"), lty=ltys, col="black")

```

Trajectoire des lambdas



Et maintenant pour les ξ_j :

```
# Délimitation de la fenêtre graphique :
ylim_rate = range(unlist(lapply(res_sim2, function(r) r$theta[,3:4])))

# Le graphique :
plot(res_sim2[[1]]$theta[,3], type="l", col=colors[1], lty=ltys[1],
      ylim=ylim_rate, xlab="Itération", ylab="Rate",
      main="Trajectoire des rate")

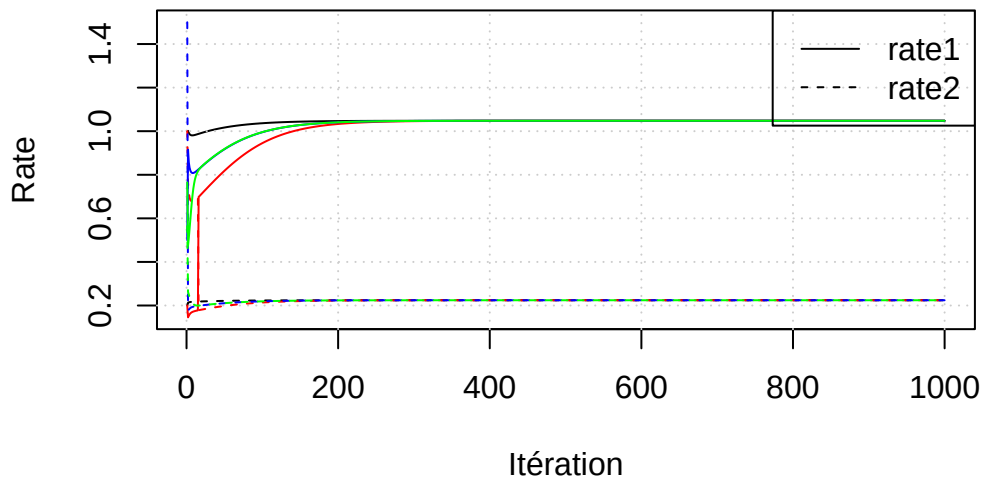
grid(col="grey80", lty=3)

lines(res_sim2[[1]]$theta[,4], col=colors[1], lty=ltys[2])

for(i in 2:length(res_sim2)){
  lines(res_sim2[[i]]$theta[,3], col=colors[i], lty=ltys[1])
  lines(res_sim2[[i]]$theta[,4], col=colors[i], lty=ltys[2])
}

legend("topright", legend=c("rate1","rate2"), lty=ltys, col="black")
```

Trajectoire des rate



Les résultats semblent assez stables, bien que cela dépende des données simulées.

On présente également les résultats obtenus pour chaque point de départ :

```
df_res = data.frame(
  start = paste0("start", 1:length(res_sim2)),
  lambda1 = NA, lambda2 = NA,
  rate1 = NA, rate2 = NA)

for(i in seq_along(res_sim2)){
  df_res$lambda1[i] = res_sim2[[i]]$lambda[1]
  df_res$lambda2[i] = res_sim2[[i]]$lambda[2]
  df_res$rate1[i] = res_sim2[[i]]$rate[1]
  df_res$rate2[i] = res_sim2[[i]]$rate[2]
}
```

df_res

	start	lambda1	lambda2	rate1	rate2
1	start1	0.3048456	0.6951544	1.047888	0.2241972
2	start2	0.3048456	0.6951544	1.047888	0.2241972
3	start3	0.3048456	0.6951544	1.047888	0.2241972
4	start4	0.3048456	0.6951544	1.047888	0.2241972

Les résultats sont identiques, peu importe le point de départ. C'est étonnant mais cela peut dépendre des données simulées (si elles sont particulièrement bien séparées), surtout que `start4` a été généré de manière aléatoire.

5. Application à un jeu de données réel

Pour l'application à un jeu de données réel, j'ai décidé d'utiliser `mgus2` du package `survival` :

```
mgus2=survival::mgus2
help(mgus2)
```

```
describe(mgus2)
```

	vars	n	mean	sd	median	trimmed	mad	min	max	range
id	1	1384	692.50	399.67	692.5	692.50	512.98	1.0	1384.0	1383.0
age	2	1384	70.42	12.17	72.0	71.25	11.86	24.0	96.0	72.0
sex*	3	1384	1.54	0.50	2.0	1.56	0.00	1.0	2.0	1.0
dxyr	4	1384	1983.50	6.37	1984.0	1983.82	5.93	1960.0	1994.0	34.0
hgb	5	1371	13.30	2.02	13.5	13.42	1.93	5.7	18.9	13.2
creat	6	1354	1.29	1.12	1.1	1.12	0.30	0.4	22.0	21.6
mspike	7	1373	1.16	0.57	1.2	1.13	0.59	0.0	3.0	3.0
ptime	8	1384	93.54	71.90	81.0	86.34	72.65	1.0	424.0	423.0
pstat	9	1384	0.08	0.28	0.0	0.00	0.00	0.0	1.0	1.0
futime	10	1384	95.80	72.38	84.0	88.82	72.65	1.0	424.0	423.0
death	11	1384	0.70	0.46	1.0	0.74	0.00	0.0	1.0	1.0

	skew	kurtosis	se
id	0.00	-1.20	10.74
age	-0.72	0.66	0.33
sex*	-0.18	-1.97	0.01
dxyr	-0.45	-0.17	0.17
hgb	-0.57	0.43	0.05
creat	9.22	120.03	0.03
mspike	0.34	-0.23	0.02
ptime	0.92	0.74	1.93
pstat	3.02	7.11	0.01
futime	0.89	0.67	1.95
death	-0.85	-1.28	0.01

C'est un jeu de données sur la gammapathie monoclonale de signification indéterminée, qui est un état pré-cancéreux. Les deux variables qui nous intéressent sont `futime`, qui représente

la durée de survie (en mois) et `death`, qui représente l'occurrence de l'évènement d'intérêt (ici le décès : `death = 1` : décès, `death = 0` : censuré)

```
# Durée de survie et indicateur de censure
t_app = mgus2$futime
d_app = mgus2$death

mean(1-d_app)
```

```
[1] 0.3041908
```

On définit également un point de départ pour les ξ_j . On suppose que les deux composantes représentent 2 échelles de durée de survie différentes (une longue et une courte), ce qui peut se justifier car une partie des patients de `mgus2` développent un stade pré-cancéreux.

```
rate_start = c(1 / quantile(t_app[d_app == 1], 0.75),
               1 / quantile(t_app[d_app == 1], 0.25))
```

Appliquons notre algorithme :

```
set.seed(98)
start_mgus2 = list(lambda=c(0.5, 0.5),
                   rate=rate_start)

res_mgus2 = my_em_exp_censored(x=t_app, d=d_app, niter=1000,
                              start=start_mgus2, verbose=FALSE)

res_mgus2$lambda ; res_mgus2$rate
```

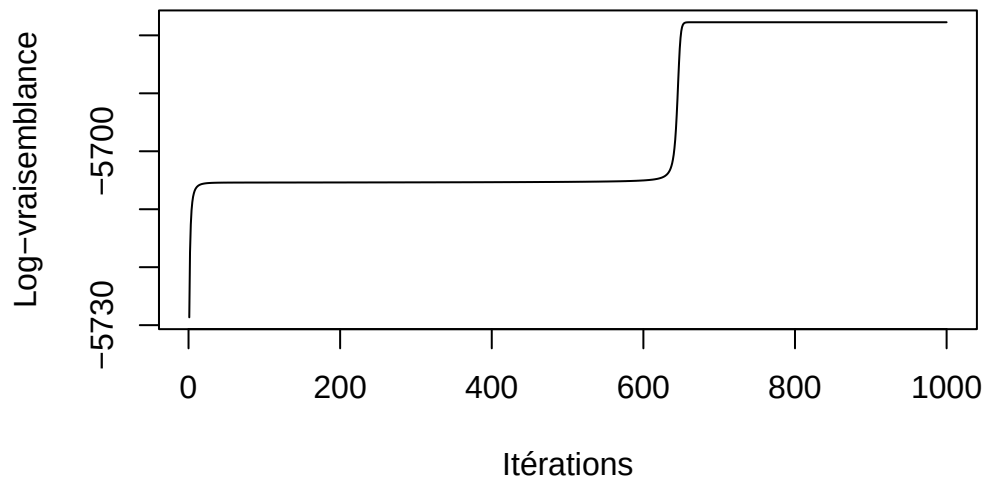
```
[1] 0.94947679 0.05052321
```

```
          75%          25%
0.006746262 0.357376796
```

Illustrons la croissance de la log-vraisemblance :

```
plot(res_mgus2$log.vrais, type="l",
     xlab="Itérations", ylab="Log-vraisemblance",
     main="Log-vraisemblance pour le jeu de données MGUS2")
```

Log-vraisemblance pour le jeu de données MGUS2



La log-vraisemblance est bien croissante et semble avoir convergé (si l'on fait 10 000 itérations elle converge toujours).

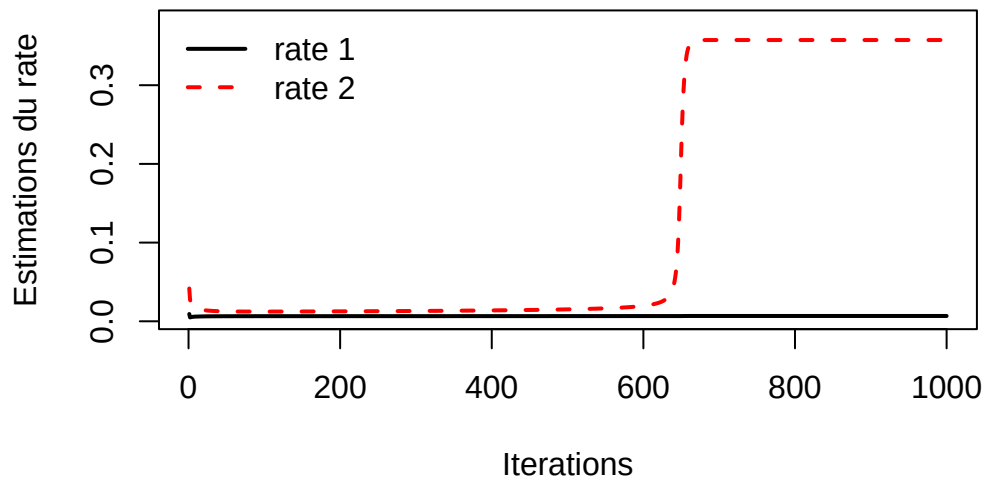
Concernant les estimations obtenues et leurs trajectoires :

```
plot(1:nrow(res_mgus2$theta), res_mgus2$theta[, 3], type = "l", lwd = 2,
     ylim = c(0.005, 0.38),
     xlab = "Iterations",
     ylab = "Estimations du rate",
     main = "EM pour MGUS2")

lines(1:nrow(res_mgus2$theta), res_mgus2$theta[, 4],
      col = "red", lty = 2, lwd = 2)

legend("topleft",
      legend = c("rate 1", "rate 2"), #, "lambda 1"),
      col = c("black", "red"), #, "green3"),
      lty = c(1, 2), #, 3),
      lwd = 2,
      bty = "n")
```

EM pour MGUS2

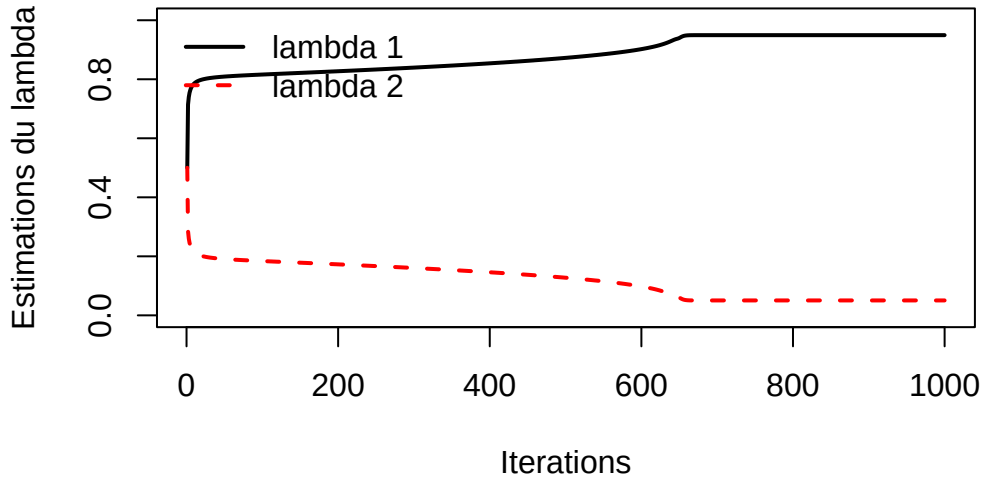


```
plot(1:nrow(res_mgus2$theta), res_mgus2$theta[, 1], type = "l", lwd = 2,
     ylim = c(0, 1),
     xlab = "Iterations",
     ylab = "Estimations du lambda",
     main = "EM pour MGUS2")

lines(1:nrow(res_mgus2$theta), res_mgus2$theta[, 2],
      col = "red", lty = 2, lwd = 2)

legend("topleft",
      legend = c("lambda 1", "lambda 2"),#, "lambda 1"),
      col = c("black", "red"),#, "green3"),
      lty = c(1, 2),#, 3),
      lwd = 2,
      bty = "n")
```

EM pour MGUS2



L'algorithme identifie donc 2 classes de taille très différentes :

$$\hat{\lambda}_1 \approx 0.9495, \quad \hat{\lambda}_2 \approx 0.0505$$

Les taux pour les deux composantes exponentielles sont :

$$\hat{\xi}_1 \approx 0.0068, \quad \hat{\xi}_2 \approx 0.3574$$

On les interprète :

$$\frac{1}{\hat{\xi}_1} \approx 147.0588, \quad \frac{1}{\hat{\xi}_2} \approx 2.7980$$

Ainsi, on a une première classe qui contient la majorité des individus et dont la durée de survie estimée est d'environ 147 mois, et une deuxième classe minoritaire, dont les individus ont une durée de survie estimée très courte : environ 3 mois.

De part les tailles particulièrement disproportionnées des distributions, on peut légitimement s'interroger sur l'adéquation d'un mélange de deux composantes exponentielles dans ce jeu de données. Peut-être qu'un modèle à une composante serait suffisant.

6. Implémentation du St-EM paramétrique

La différence avec l'algorithme EM précédent est l'implémentation d'une étape stochastique supplémentaire, où à chaque itération k on re-simule $Z_i^k \sim \text{Mult}(1, p_i^k)$, $p_i^k = (p_{i1}^k, \dots, p_{im}^k)$. On crée ensuite des sous-ensembles χ_j^k qui contiendront les $Z_i^k = j$. Les formules de mise à jour de M. BORDES sont alors :

$$\lambda_j^{(k+1)} = \frac{\text{Card}(\chi_j^k)}{n}$$

et

$$\xi_j^{(k+1)} = \arg \max_{\xi \in \Xi} L_j(\xi)$$

$$\text{où } L_j(\xi) = \prod_{i \in \chi_j^k} f(t_i | \xi)^{d_i} \bar{F}(t_i | \xi)^{1-d_i}$$

Si on se place dans le cas d'un modèle de mélange exponentiel à deux composantes, on a pour $j \in \{1, 2\}$, $f(t_i | \xi_j) = \xi_j e^{-\xi_j t_i}$ et $\bar{F}(t_i | \xi_j) = e^{-\xi_j t_i}$. Alors,

$$\begin{aligned} L_j(\xi) &= \prod_{i \in \chi_j^k} [\xi_j e^{-\xi_j t_i}]^{d_i} [e^{-\xi_j t_i}]^{1-d_i} \\ &= \xi_j^{\sum_{i \in \chi_j^k} d_i} \exp \left(-\xi_j \sum_{i \in \chi_j^k} t_i \right) \end{aligned}$$

On obtient comme formule de mise à jour pour les $\xi_j^{(k+1)}$:

$$\xi_j^{(k+1)} = \frac{\sum_{i \in \chi_j^k} d_i}{\sum_{i \in \chi_j^k} t_i}$$

Implémentons cet algorithme :

```
my_stem_exp_censored = function(x, d, niter, start, trace=TRUE, verbose){
  n = length(x)
  lambda = start$lambda
  rate = start$rate
  K = length(lambda)
  eta = matrix(NA, nrow=n, ncol=K)
  theta = NULL
  log.vrais=c()
```

```

iterations = 1:niter

if(trace){
  theta=matrix(NA,niter,2*K)
}

for(iter in 1:niter){

  if(trace){
    theta[iter,] = c(lambda,rate)
  }

  # Mise à jour des  $p_{ij}^k$ 
  for(k in 1:K){
    eta[,k] = lambda[k] * (d*dexp(x, rate=rate[k]) +
      (1-d)*pexp(x, rate = rate[k], lower.tail = FALSE))
  }

  eta=eta/rowSums(eta)

  # Etape stochastique

  Z = apply(eta, 1, function(p){
    sample(1:K, size=1, prob=p)})

  # Mise à jour des paramètres

  lambda = tabulate(Z, nbins=K) / n

  for(k in 1:K){
    j=which(Z==k)
    if(length(j) > 0 && sum(x[j]) > 0){
      rate[k] = sum(d[j]) / sum(x[j])
    }
  }

  log.vrais[iter]=vrais.comp(x,d,lambda,rate)

  if(verbose) print(c(iter,lambda,rate))

}
return(list(lambda=lambda, rate=rate, theta=theta,

```

```

        log.vrais=log.vrais, iteration = iterations))
}

```

Un problème que j'ai fréquemment rencontré avec cet algorithme stochastique était que parfois on atteignait un état dégénéré, dans lequel une certaine classe pouvait ne plus être du tout représentée ($\lambda_1 = 0$ par exemple), ou qu'un ξ_j soit égal à 0. Ce problème est mentionné par McLachlan & Peel (2) dans la section 2.5.

J'ai donc du mettre des valeurs planchers de sécurité pour ces paramètres, de sorte à ce que même si un paramètre vaut 0 à un moment donné, cela ne bloque pas la suite de l'algorithme :

```

my_stem_exp_censored = function(x, d, niter, start, trace=TRUE, verbose){
  n = length(x)
  lambda = start$lambda
  rate = start$rate
  K = length(lambda)
  eta = matrix(NA, nrow=n, ncol=K)
  theta = NULL
  log.vrais=c()
  iterations = 1:niter

  if(trace){
    theta=matrix(NA,niter,2*K)
  }

  # Sécurité
  lambda_min = 1e-6
  rate_min = 1e-4
  n_min = 5
  Z = sample(1:K, size = n, replace = TRUE, prob = lambda)
  # --

  for(iter in 1:niter){

    if(trace){
      theta[iter,] = c(lambda,rate)
    }

    # Mise à jour des p_ij^k
    for(k in 1:K){
      eta[,k] = lambda[k] * (d*dexp(x, rate=rate[k]) +

```

```

        (1-d)*pexp(x, rate = rate[k], lower.tail = FALSE))
    }

    eta=eta/rowSums(eta)

    # Etape stochastique

    Z_old = Z

    Z = apply(eta, 1, function(p){
        sample(1:K, size=1, prob=p)})

    if(any(tabulate(Z, nbins=K) < n_min)){
        Z = Z_old
    }

    # Mise à jour des paramètres

    lambda = tabulate(Z, nbins=K) / n
    lambda = pmax(lambda, lambda_min)
    lambda = lambda/sum(lambda)

    for(k in 1:K){
        j=which(Z==k)
        if(length(j) > 0 && sum(x[j]) > 0){
            rate_maj = sum(d[j]) / sum(x[j])
            rate[k] = max(rate_maj, rate_min)
        }
    }

    log.vrais[iter]=vrais.comp(x,d,lambda,rate)

    if(verbose) print(c(iter,lambda,rate))

}
return(list(lambda=lambda, rate=rate, theta=theta,
            log.vrais=log.vrais, iteration = iterations))
}

```

```

set.seed(98)
# Application de l'algorithme St-EM :
res_st1=my_stem_exp_censored(x=sim1[,1], d=sim1[,2],

```

```

                                niter=1000, start=start, verbose=FALSE)
res_st2=my_stem_exp_censored(x=sim2[,1], d=sim2[,2],
                                niter=1000, start=start, verbose=FALSE)

```

Les résultats obtenus s'interprètent différemment. En effet, les résultats ne convergent pas vers une valeur précise des paramètres (ce qui est le cas de l'algorithme EM) mais convergent au sens markovien du terme vers une loi stationnaire. On ne peut donc pas interpréter la dernière valeur de l'algorithme comme étant la valeur estimée du paramètre.

Pour cela, Celeux, Chauveau et Diebolt (1995) ont proposé différentes méthodes (3). La première et la plus simple est de considérer les premières itérations de l'algorithme comme étant un "échauffement" (warm-up), puis d'utiliser la moyenne de toutes les observations suivantes comme étant un estimateur des paramètres. Cela donne ici :

```

warm_up=300

theta_hat = colMeans(res_st1$theta[(warm_up+1):1000, ])
theta_hat

```

```
[1] 0.1323286 0.8676714 4.4555472 0.2559558
```

```

theta_hat2 = colMeans(res_st2$theta[(warm_up+1):1000, ])
theta_hat2

```

```
[1] 0.3171014 0.6828986 1.0480098 0.2211547
```

```

theta = res_st1$theta

plot(1:nrow(theta), theta[, 3], type = "l", lwd = 2,
     ylim = c(0, 1.5),
     xlab = "iterations",
     ylab = "estimates",
     main = "St-EM pour mélange exponentiel,
           (n=200)")

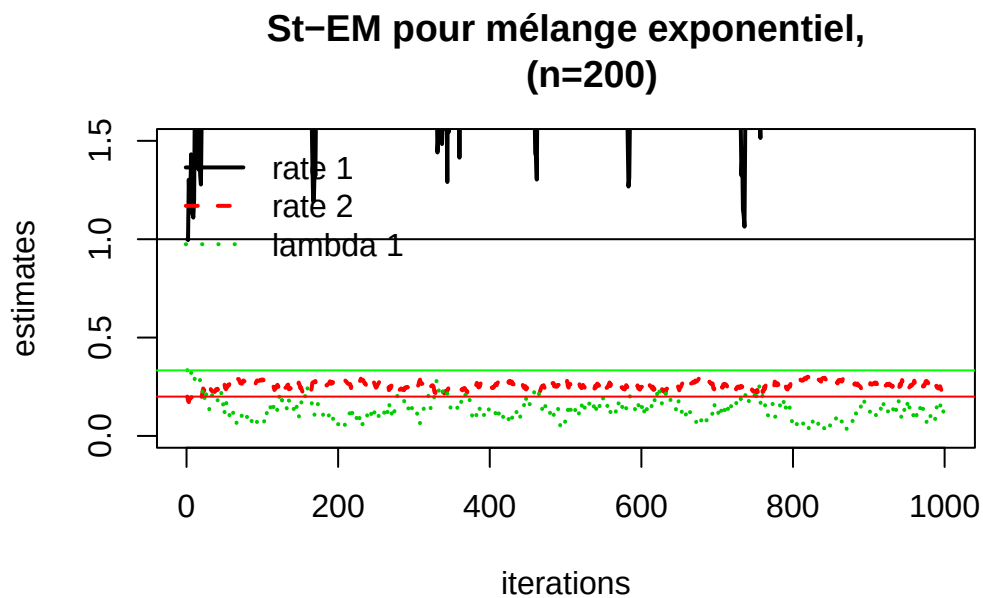
lines(1:nrow(theta), theta[, 4], col = "red", lty = 2, lwd = 2)
lines(1:nrow(theta), theta[, 1], col = "green3", lty = 3, lwd = 2)

abline(h = 1)
abline(h=.2, col = "red")

```

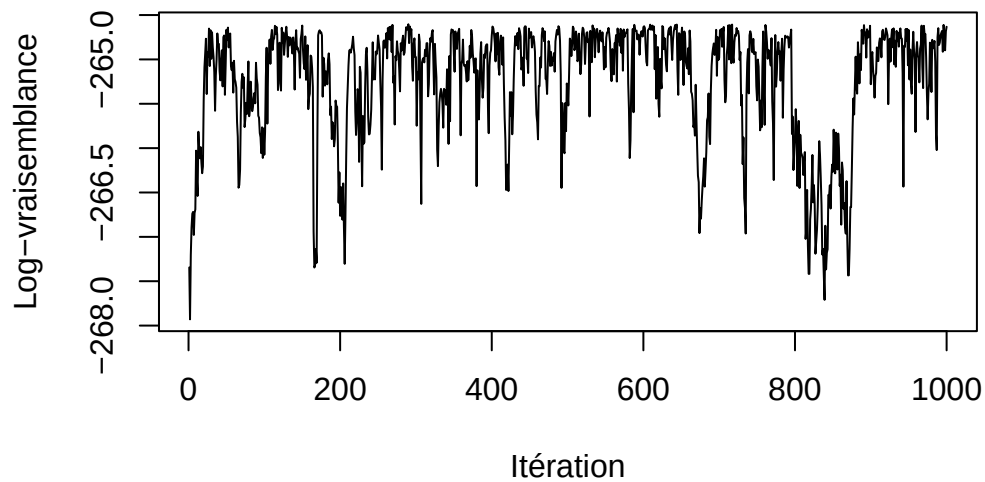
```
abline(h=1/3, col="green")

legend("topleft",
      legend = c("rate 1", "rate 2", "lambda 1"),
      col = c("black", "red", "green3"),
      lty = c(1, 2, 3),
      lwd = 2,
      bty = "n")
```



```
plot(res_st1$log.vrais, type="l",
      xlab="Itération", ylab="Log-vraisemblance",
      main = "Log-vraisemblance dans le 1er jeu simulé,  
(n=200)")
```

Log-vraisemblance dans le 1er jeu simulé, (n=200)



Pour le deuxième jeu de données :

```
theta = res_st2$theta

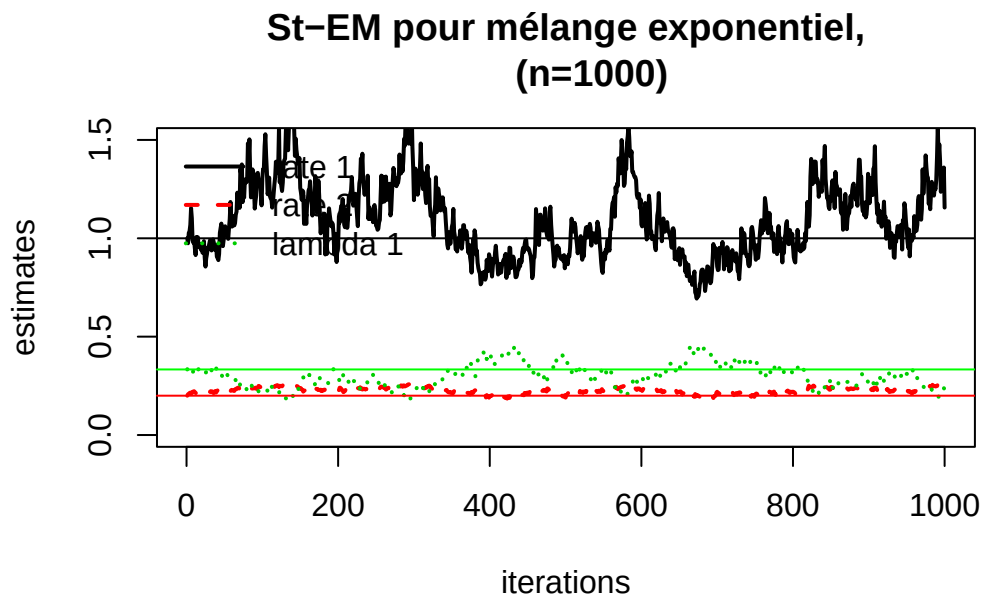
plot(1:nrow(theta), theta[, 3], type = "l", lwd = 2,
     ylim = c(0, 1.5),
     xlab = "iterations",
     ylab = "estimates",
     main = "St-EM pour mélange exponentiel,
     (n=1000)")

lines(1:nrow(theta), theta[, 4], col = "red", lty = 2, lwd = 2)
lines(1:nrow(theta), theta[, 1], col = "green3", lty = 3, lwd = 2)

abline(h = 1)
abline(h=.2, col = "red")
abline(h=1/3, col="green")

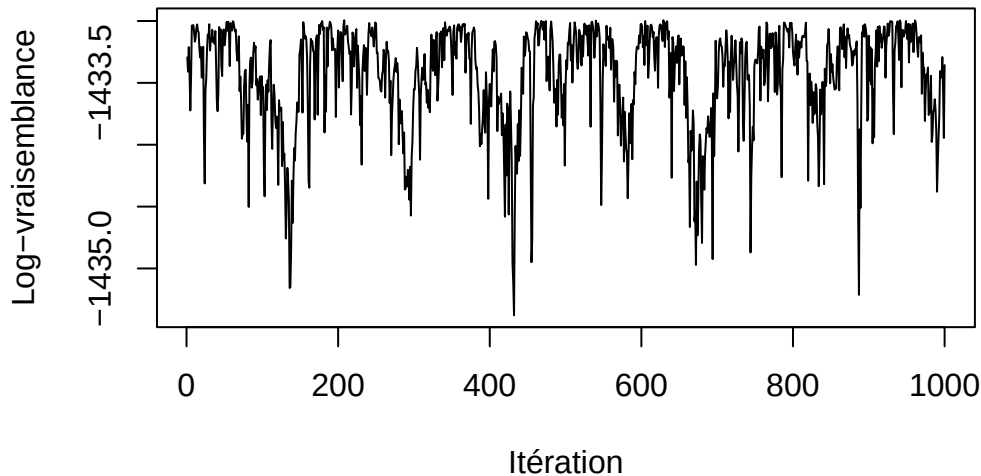
legend("topleft",
      legend = c("rate 1", "rate 2", "lambda 1"),
      col = c("black", "red", "green3"),
      lty = c(1, 2, 3),
      lwd = 2,
```

```
bty = "n")
```



```
plot(res_st2$log.vrais, type="l",  
      xlab="Itération", ylab="Log-vraisemblance",  
      main = "Log-vraisemblance pour le deuxième jeu simulé  
(n=1000)")
```


Log-vraisemblance pour le deuxième jeu simulé (n=1000)



On va maintenant étudier le comportement de l'algorithme sur le deuxième jeu simulé (donc avec $n = 1000$) en fonction du point de départ. On conserve les mêmes points de départ que ceux initialisés dans la question 4.

```
set.seed(98)
res_st212 = my_stem_exp_censored(x=sim2[,1], d=sim2[,2], niter=1000,
                                start=start2, verbose=FALSE)
res_st213 = my_stem_exp_censored(x=sim2[,1], d=sim2[,2], niter=1000,
                                start=start3, verbose=FALSE)
res_st214 = my_stem_exp_censored(x=sim2[,1], d=sim2[,2], niter=1000,
                                start=start4, verbose=FALSE)

res_st_sim2 = list(res_st2, res_st212, res_st213, res_st214)
```

Les résultats sont :

```
theta_st_list = lapply(res_st_sim2, function(r){
  t(apply(r$theta, 1, function(v){
    lambda = v[1:2]
    rate = v[3:4]
    o = order(lambda)
    c(lambda[o], rate[o])
  })
})
```

```

    )))
  })

```

```

ylim_lam = range(unlist(lapply(res_st_sim2, function(r) r$theta[,1:2])))

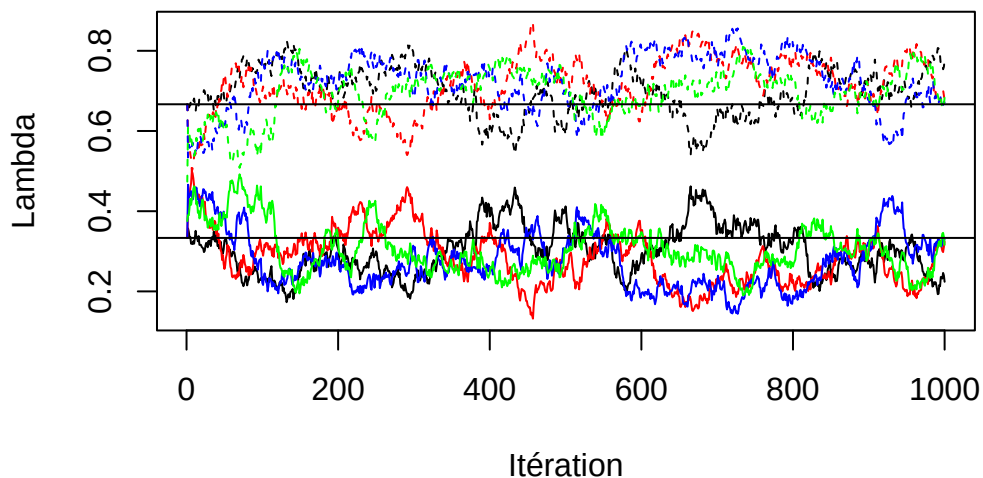
plot(theta_st_list[[1]][,1], type="l", col=colors[1], lty=ltys[1],
      ylim=ylim_lam, xlab="Itération", ylab="Lambda",
      main="Trajectoire des lambdas - St-EM sim2")
lines(theta_st_list[[1]][,2], col=colors[1], lty=ltys[2])

for(i in 2:length(theta_st_list)){
  lines(theta_st_list[[i]][,1], col=colors[i], lty=ltys[1])
  lines(theta_st_list[[i]][,2], col=colors[i], lty=ltys[2])
}

abline(h=1/3)
abline(h=2/3)

```

Trajectoire des lambdas – St-EM sim2



```

ylim_rate = range(unlist(lapply(res_st_sim2, function(r) r$theta[,3:4])))

plot(theta_st_list[[1]][,3], type="l", col=colors[1], lty=ltys[1],
      ylim=ylim_rate, xlab="Itération", ylab="Rate",

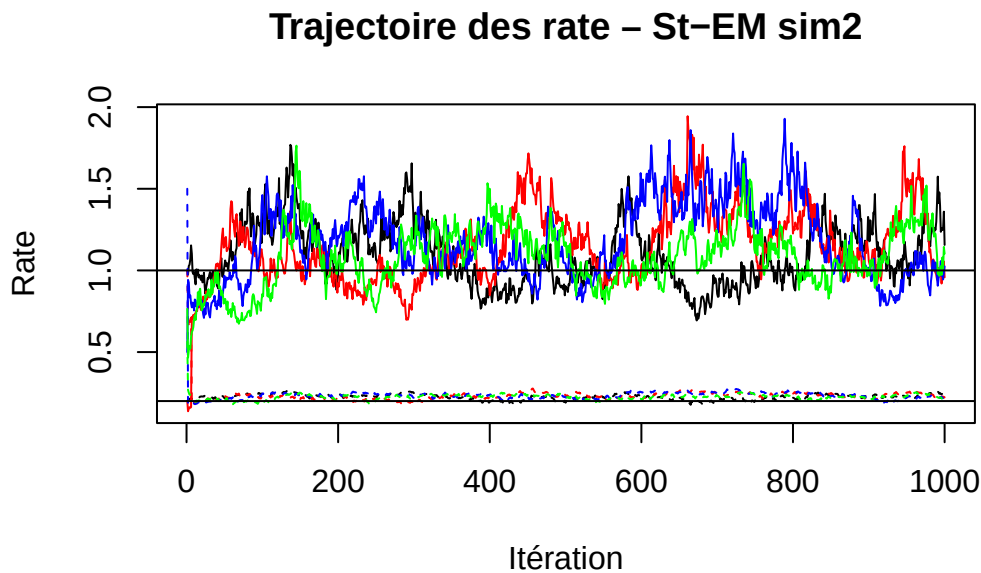
```

```

    main="Trajectoire des rate - St-EM sim2")
lines(theta_st_list[[1]][,4], col=colors[1], lty=ltys[2])

for(i in 2:length(theta_st_list)){
  lines(theta_st_list[[i]][,3], col=colors[i], lty=ltys[1])
  lines(theta_st_list[[i]][,4], col=colors[i], lty=ltys[2])
}
abline(h=1)
abline(h=.2)

```



Pour avoir quelque chose de facilement interprétable, on fait la moyenne des valeurs post-échauffement :

```

mean_post_warmup = function(theta, warm_up){
  theta_post = theta[(warm_up+1):nrow(theta), ] # ce qui suit le warmup
  lambda_mean = colMeans(theta_post[,1:2]) # moyenne pour les lambda
  rate_mean = colMeans(theta_post[,3:4]) # moyenne pour les rate
  list(lambda=lambda_mean, rate=rate_mean)
}

means_starts = lapply(theta_st_list, mean_post_warmup, warm_up=warm_up)

df_means = do.call(rbind, lapply(seq_along(means_starts), function(i){

```

```

cbind(start = i, t(means_starts[[i]]$lambda),
      t(means_starts[[i]]$rate))
)))

colnames(df_means) = c("start", "lambda1", "lambda2", "rate1", "rate2")

df_mean = as.data.frame(df_means)

df_mean

```

```

  start  lambda1  lambda2  rate1  rate2
1      1 0.3171014 0.6828986 1.048010 0.2211547
2      2 0.2597986 0.7402014 1.217065 0.2367105
3      3 0.2682229 0.7317771 1.201130 0.2345516
4      4 0.2891071 0.7108929 1.124028 0.2281186

```

```
cat("Le lambda1 moyen estimé est :", mean(df_mean[,2]), "\n")
```

Le lambda1 moyen estimé est : 0.2835575

```
cat("Le lambda2 moyen estimé est :", mean(df_mean[,3]), "\n")
```

Le lambda2 moyen estimé est : 0.7164425

```
cat("Le rate1 moyen estimé est :", mean(df_mean[,4]), "\n")
```

Le rate1 moyen estimé est : 1.147558

```
cat("Le rate2 moyen estimé est :", mean(df_mean[,5]), "\n")
```

Le rate2 moyen estimé est : 0.2301339

On identifie une légère variation des estimations selon le point de départ. **start1**, qui contient les vraies valeurs, nous amènent à de meilleures estimations. Les estimations sont cependant plus proche de la réalité pour les ξ_j que celles produites par l'algorithme EM classique.

Toutefois, les estimations ne contiennent pas de valeurs particulièrement extrêmes ou éloignées de la réalité. L'algorithme St-EM est relativement stable.

7. Application à un jeu de données réel

On ré-utilise le même jeu de données `mgus2` :

```
set.seed(98)
res_st_mgus2 = my_stem_exp_censored(x=t_app, d=d_app, niter=1000,
                                   start=start_mgus2, verbose=FALSE)
```

On peut afficher les trajectoires des paramètres estimés :

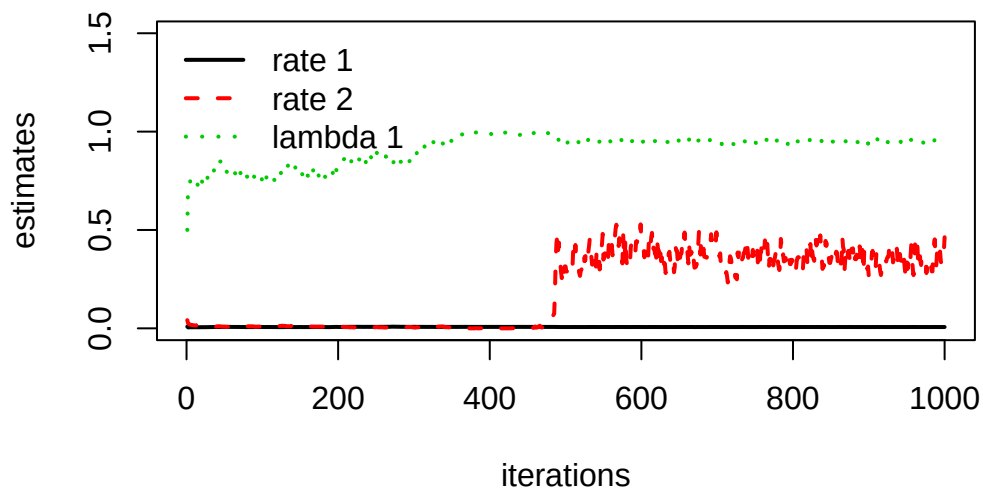
```
theta_mgus2 = res_st_mgus2$theta

plot(1:nrow(theta_mgus2), theta_mgus2[, 3],
     type = "l", lwd = 2,
     ylim = c(0, 1.5),
     xlab = "iterations",
     ylab = "estimates",
     main = "St-EM pour les données MGUS2")

lines(1:nrow(theta_mgus2), theta_mgus2[, 4],
      col = "red", lty = 2, lwd = 2)
lines(1:nrow(theta_mgus2), theta_mgus2[, 1],
      col = "green3", lty = 3, lwd = 2)

legend("topleft",
      legend = c("rate 1", "rate 2", "lambda 1"),
      col = c("black", "red", "green3"),
      lty = c(1, 2, 3),
      lwd = 2,
      bty = "n")
```

St-EM pour les données MGUS2



```
theta_hat_app = colMeans(res_st_mgus2$theta[(warm_up+1):1000, ])  
cat("Le lambda1 estimé est :", theta_hat_app[1], "\n")
```

Le lambda1 estimé est : 0.9569798

```
cat("Le lambda2 estimé est :", theta_hat_app[2], "\n")
```

Le lambda2 estimé est : 0.04302023

```
cat("Le rate1 estimé est :", theta_hat_app[3], "\n")
```

Le rate1 estimé est : 0.006919777

```
cat("Le rate2 estimé est :", theta_hat_app[4], "\n")
```

Le rate2 estimé est : 0.2715428

L'EM stochastique identifie donc 2 classes de taille très différentes (encore plus que l'algorithme EM):

$$\hat{\lambda}_1 \approx 0.9570, \quad \hat{\lambda}_2 \approx 0.0430$$

Les taux pour les deux composantes exponentielles sont :

$$\hat{\xi}_1 \approx 0.0069, \quad \hat{\xi}_2 \approx 0.2715$$

On les interprète :

$$\frac{1}{\hat{\xi}_1} \approx 144.9275, \quad \frac{1}{\hat{\xi}_2} \approx 3.2832$$

Ainsi, on a une première classe qui contient la majorité des individus et dont la durée de survie estimée est d'environ 145 mois, et une deuxième classe minoritaire, dont les individus ont une durée de survie estimée très courte : environ 3 mois.

On conclut encore une fois à la possible inadéquation d'un modèle de mélange exponentiel à deux composantes de ce jeu de données.

Conclusion et perspectives

Les algorithmes EM et St-EM paramétriques se révèlent adaptés pour explorer la structure des données avec variable qualitative latente et les données manquantes. Comme attendu, le Stochastic EM semble plus robuste. Bien qu'il soit plus coûteux en temps de calcul, les résultats sont plus robustes : il permet de faire face à des structures plus complexes, et se détacher des maximums locaux parfois atteint l'EM.

Concernant les perspectives envisageables, on peut imaginer travailler avec des modèles plus complexes, des mélanges à plus de deux composantes ou avec un taux de censure plus élevé.

On pourrait aussi améliorer certaines implémentations faites, notamment la manière d'éviter les cas dégénérés dans le Stochastic EM. La littérature suggère notamment l'utilisation de lois à priori conjuguées (ici donc des Gamma). Le choix des points de départ peut certainement aussi être amélioré.

Bibliographie

1. Geffray S. UE GLM. 2025.
2. McLachlan G, Peel D. Finite Mixture Models [Internet]. 1st ed. Wiley; 2000 [cited 2025 Dec 31]. (Wiley Series in Probability and Statistics). Available from: <https://onlinelibrary.wiley.com/doi/book/10.1002/0471721182>
3. Celeux G, Chauveau D, Diebolt J. On Stochastic Versions of the EM Algorithm [Internet]. INRIA; 1995 [cited 2025 Dec 28]. Report No.: RR-2514. Available from: <https://inria.hal.science/inria-00074164>
4. Bordes L, Chauveau D. [Some Algorithms to Fit some Reliability Mixture Models under Censoring](#). In: Lechevallier Y, Saporta G, editors. Proceedings of COMPSTAT'2010. Heidelberg: Physica-Verlag HD; 2010. p. 243–52.
5. Bordes L, Chauveau D. Stochastic EM algorithms for parametric and semiparametric mixture models for right-censored lifetime data. Computational Statistics [Internet]. 2016 Dec [cited 2025 Dec 29];31(4):1513–38. Available from: <http://link.springer.com/10.1007/s00180-016-0661-7>
6. Stephens M. Dealing With Label Switching in Mixture Models. Journal of the Royal Statistical Society Series B: Statistical Methodology [Internet]. 2000 Nov [cited 2025 Dec 31];62(4):795–809. Available from: <https://academic.oup.com/jrsssb/article/62/4/795/7083253>